

ตอบข้อวิจารณ์แผนภาพ Robot aggregate

เอกสารนี้ตอบข้อวิจารณ์ 40 ข้อที่มีต่อแผนภาพ Robot aggregate (แผนภาพ 3 ในรีวิวลสถาปัตยกรรม Nsc-Unity) ทุกข้อถูกเทียบกับโค้ดจริงใน branch ye1mee ก่อนตัดสินใจ ตัดสิน ข้อที่ถูกได้แก้แล้วในรีวิว (ARCHITECTURE_REVIEW.th.md) และในแผนภาพ (uml/) ข้อที่เห็นต่างมีเหตุผลและหลักฐานกำกับ

ตัวเลขแบบ ไฟล์.cs:บรรทัด อ้างอิงโค้ดใน branch นี้ ส่วนคำว่า "แผนภาพ N" หมายถึงลำดับใหม่ในรีวิวที่ปรับแล้ว (มีทั้งหมด 14 แผนภาพ)

สารบัญ

A. สรุป	3
B. บริบทที่ใช้ตัดสินใจ	3
C. สิ่งที่เกี่ยวข้อง.....	4
D. ตอบทีละข้อ.....	4
E. ข้อที่เห็นต่าง อธิบายเพิ่ม	9
E1. อินเทอร์เฟซ support แบบรวม (ข้อ 2)	9
E2. state ของลำตัวควรมีมากขึ้น (ข้อ 23)	9
E3. ชั้น command แยก และเมธอด Server* (ข้อ 20, 21).....	10
E4. ชั้นฟิสิกส์แยก และชื่อ Controller (ข้อ 9, 37, 38)	10
E5. topology แบบยืดหยุ่น และ capability (ข้อ 16, 17, 39).....	10
E6. Robot รับงานย่อย และ FromCollider (ข้อ 4, 5)	10
E7. แยก LegController เป็นหลายคลาส (ข้อ 10).....	11
F. ปัญหาที่เจอระหว่างตรวจ.....	11
F1. ลำตัวยังนับขาที่หลุดเป็นเท้า (High).....	11
F2. ค่าแรงป็นถูกเขียนหับ (Medium)	11
F3. respawn ฟังก์ joint ที่แคชไว้ตอนเริ่มฉาก (Medium).....	11
F4. RPC ของแขนขาไม่ได้จำกัดเจ้าของ (Medium)	12
G. ลำดับความสำคัญหลังตรวจโค้ด	12
H. เรื่องที่ทีมต้องตัดสินใจ	12

A. สรุป

ผล	จำนวน	ข้อ
ยอมรับ และแก้แล้ว	9	6, 7, 11, 13, 25, 27, 29, 33, 34
ยอมรับบางส่วน	11	1, 2, 3, 8, 10, 14, 19, 26, 30, 31, 40
เห็นต่าง	13	4, 5, 9, 12, 16, 17, 18, 20, 21, 23, 37, 38, 39
ชี้แจง (แผนภาพอื่นในชุดตอบไว้แล้ว หรือแก้แค่การวาด)	6	15, 22, 24, 32, 35, 36
อ่านแผนภาพผิด แต่ประเด็นจริงในโค้ด	1	28

ข้อวิจารณ์ถูกในเรื่องที่สำคัญที่สุด คือวงจรชีวิตของแขนขาที่หลุดและที่อยู่ของกฎการล้ม ตอนตรวจโค้ดเพื่อตอบข้อวิจารณ์นี้เจอปัญหาที่น่าจะเป็นบั๊กจริง 4 จุด (หัวข้อ F) ซึ่งสนับสนุนข้อวิจารณ์ และทำให้รีวิวเปลี่ยนความเห็นเดิมหนึ่งเรื่อง คือจากเดิมที่บอกว่าการแยก `FallRules` ไม่บังคับ ตอนนี้แนะนำให้ทำ

ส่วนข้อเสนอที่เพิ่มความทั่วไป ได้แก่ อินเทอร์เฟซ `support` แบบรวม `state` ของลำตัวที่มากขึ้น ชั้น `command` แยก ชั้น `ฟิสิกส์` แยก `topology` แบบยืดหยุ่น และระบบ `capability` ไม่เข้ากับเกมนี้ และมีหลักฐานในโค้ดว่าบางข้อจะทำให้กฎของเกมแสดงออกได้แยลง รายละเอียดอยู่ในหัวข้อ E

B. บริบทที่ใช้ตัดสิน

- **แผนภาพ 3 เป็นหนึ่งในชุด** ข้อวิจารณ์ดูแผนภาพนี้ภาพเดียว แต่หลายข้อถูกตอบไว้แล้วในภาพอื่น: การไหลของดาเมจ (แผนภาพ 6 และ 9) การควบคุมและ RPC (แผนภาพ 5) การหลุดของแขนขา (แผนภาพ 11) และ data flow (แผนภาพ 14) รอบนี้เพิ่มภาพที่ขาดจริงอีก 3 ภาพ (แผนภาพ 4, 12, 13)
- **เกณฑ์ของรีวิว** สถาปัตยกรรมต้องพอดีกับขนาดของเกม อินเทอร์เฟซใหม่ต้องมีผู้ใช้ที่ไม่เกี่ยวข้องอย่างน้อยสองแบบ (รีวิวข้อ O กฎ 8) และชั้นใหม่ต้องตัดความซ้ำหรือบั๊กที่มีอยู่จริง (กฎ 17)
- **ข้อเท็จจริงของเกมที่ส่งผลต่อการตัดสิน**
 - ผู้เล่น 4 คนคุมแขนขาคนละชั้น topology 4 ชั้นจึงเป็นแก่นของเกม ไม่ใช่ข้อจำกัดที่บังเอิญ
 - ฟิสิกส์ของหุ่นรันบน server เท่านั้น `NetworkTransform` ทั้ง 15 ตัวใน `RobotContainer.prefab` เป็น `AuthorityMode Server` และ ownership บอกได้ว่าใครส่ง input (`RespawnManager.cs:186-190`)
 - กติกาหลักของการล้มคือ "สองคนเผลอก้าวพร้อมกัน" (`TorsoMovement.cs:245-258`) ซึ่งเป็นกฎของชาโดโดยเฉพาะ

C. สิ่งที่แก้แล้ว

แผนภาพ

- แผนภาพ 3 (วาดใหม่) Robot aggregate และวงจรชีวิตของแขนขา: RobotLimb คือ LimbController และเป็นที่อยู่ของกติกาสองชั้น อีเวนต์บอกว่าขึ้นใหม่ (StateChanged(RobotLimb)) เพิ่ม RobotBodyDamageRelay สำหรับดาเมจที่ลำตัว SavedJointSettings เป็น struct และมีโน้ตบอกว่าหลุดแล้วอะไรยังอยู่
- แผนภาพ 4 (ใหม่) การทรงตัวของลำตัว: TorsoBalance ไม่มีรายชื่อแขนขา อ่านแขนขาทั้ง 4 ชั้นจาก Robot มาเป็น BalanceInput (ขาที่หลุดยังนับแบบเดิมเป็นค่าเริ่มต้น) แล้วให้ FallRules คืน FallReason TorsoState เหลือ {Standing, Ragdoll} และ stress ถูกระบุว่า {server only}
- แผนภาพ 5 (วาดใหม่) การควบคุมแขนขา: LimbController เป็นคลาสฐานของแขนและขา แยก ArmGrip ขามี LegMode และ RPC ทุกตัวเป็น owner-only
- แผนภาพ 11 (เขียนใหม่) เลือดหมด ขาหลุด หุ่นล้ม แล้วดึงกลับมาต่อ พร้อมตารางวงจรชีวิตของชั้นที่หลุด
- แผนภาพ 12 (ใหม่) ก้าวเท้าหนึ่งครั้ง จาก input ถึงจอทุกคน (ใครเป็น authority)
- แผนภาพ 13 (ใหม่) respawn และลำดับการรีเซ็ต
- แผนภาพ 15 (ใหม่) พฤติกรรมที่ผู้เล่นเห็น: state machine ของลำตัว แขนขา และแมตซ์ พร้อมตัวเลขจาก prefab ที่ใช้จริง เป็นสัญญาณว่ารีแพกเตอร์ต้องไม่เปลี่ยนสิ่งที่ผู้เล่นรู้สึก ทุกแผนภาพมีป้ายบอกว่าเป็นโค้ดปัจจุบันหรือข้อเสนอ และกล่องคลาสที่เสนอมีป้าย {เดิม: X} บอกว่ามาจากคลาสไหน

ข้อความในรีวิว

- ข้อ B เพิ่มแถว 25-28 (ปัญหาที่เจอในหัวข้อ F ของเอกสารนี้)
- ข้อ C ปรับคำแนะนำของ TorsoMovement, PlayerFootForRobot/PlayerHandMovement และ JointPullAndReconnect
- ข้อ D เพิ่มชื่อใหม่ (LimbController, ArmGrip, FallRules, FootContact, LegMode, FallReason) เปลี่ยน ServerRegisterFootJump เป็น ServerApplyJump และลด TorsoState เหลือสองค่า
- ข้อ E, F, H, K, L, N ปรับให้ตรงกับโครงใหม่
- ข้อ G เพิ่มตาราง "ใครเป็น authority ของอะไร"
- ข้อ M เพิ่มชั้น 2b (การทรงตัวและวงจรชีวิตของแขนขา) พร้อมเทสที่ต้องผ่าน
- ข้อ O เพิ่มกฎ 15 (ชั้นส่วนไม่ลงทะเบียนกันเอง) และกฎ 16 (state ที่มีเหตุผลเปลี่ยนผ่านเมธอดที่รับเหตุผล)

D. ตอบทีละข้อ

#	ข้อวิจารณ์	ผล	คำตอบและสิ่งที่แก้
1	TorsoBalance เป็น God Object	ยอมรับบางส่วน	ถูกว่ากฎการล้มไม่ควรปนกับแรงพุง TorsoMovement ตอนนี้มีกฎ 5 ข้อ ตัวจับเวลา 4 ตัว แรงพุง การกระโดดสองคน และการลุก และ state ถูกเขียนจากภายนอกอีก 2 คลาส (JointPullAndReconnect.cs:183, BlackHoleProjectile.cs:202) แก้: แยก FallRules (คลาส C# ธรรมดา) รับ BalanceInput คืน FallReason และให้ ServerKnockDown(reason) เป็นทางเดียวที่คลาสอื่นสั่งล้มได้ แรงพุง การกระโดด และการลุกอยู่ที่ TorsoBalance ต่อ เพราะใช้ rigidbody และ

#	ข้อวิจารณ์	ผล	คำตอบและสิ่งที่แก้
			state ตัวเดียวกันทุก tick แยกต่อก็ได้แค่คลาสที่ต้องคุยกันทุก tick (แผนภาพ 4)
2	List<LegController> / List<ArmController> ผูกแน่นเกินไป ควรใช้ ISupportProvider / IGrabProvider	ยอมรับบางส่วน	เอารายชื่อออกแล้ว ลำตัวอ่านแขนขาจาก Robot ทุก tick (ดูข้อ 28-29) แต่ไม่ใช่อินเทอร์เฟซ support แบบรวม เพราะกฎแยกมือจับกับขาขึ้นคนละแบบ: มือที่จับของหนึ่งกันได้ทุกกฎรวม stress ส่วนขาขึ้นไม่กัน stress (TorsoMovement.cs:157 เทียบกับ TorsoMovement.cs:224) และกฎหลักเป็นของขาโดยเฉพาะ (TorsoMovement.cs:249) อินเทอร์เฟซรวมจะลบความต่างนี้ทิ้ง และแต่ละอินเทอร์เฟซจะมีผู้ implement ตัวเดียว ใช้ struct FootContact กับ property อ่านอย่างเดียวแทน (หัวข้อ E1)
3	TorsoBalance มีทั้ง network state, physics, rules และ transition	ยอมรับบางส่วน	กฎและการตัดสินใจย้ายไป FallRules ซึ่งขึ้นเหตุผลมาด้วย คำถาม "ทำไมล้ม" จึงตอบได้จาก log ไม่ต้องใส่ stress, support, physics เอง ส่วน NetworkVariable ต้องอยู่บน NetworkBehaviour อยู่แล้ว การให้มันอยู่กับคอมโพเนนต์ที่เป็นเจ้าของ rigidbody ของลำตัวคือแบบปกติของ NGO
4	Robot รับงานเยอะ และ FromCollider ทำให้ Unity physics รั่วเข้า domain	เห็นต่าง	โปรเจกต์นี้ไม่มีและไม่ควรมีชั้น domain ที่แยกจาก Unity หุ่นเป็น MonoBehaviour ที่เป็น aggregate root และเมธอดทั้ง 5 เป็นงานของหุ่นทั้งตัว FromCollider มาแทนการหาหุ่นหลายแบบที่มีอยู่ เช่น RobotBodyCheck ที่ใช้ transform.root (NetworkCheck.cs:41) ส่วนดาเมจไม่ผ่าน Robot อยู่แล้ว เส้นทางคือ Collision → LimbStrike → DamageRouter → IDamageable ตรงตามข้อวิจารณ์เสนอ (แผนภาพ 6) (หัวข้อ E6)
5	Robot ไม่ควรเป็นที่รวม event ทุกตัว ควรมีระบบกลาง	เห็นต่าง	Robot คือระบบกลางของชิ้นส่วนตัวเอง คลาสใหม่ที่รับอีเวนต์แล้วอัปเดต aggregate คือของเดิมที่เปลี่ยนชื่อและเพิ่มอีกหนึ่งทอด Robot ฟังแค่ StateChanged ของแขนขา 4 ชิ้น แล้วประกาศต่อเป็น LimbStateChanged(limb) ให้ HUD และกติกาของโหมด
6	RobotLimb ยังไม่เป็นตัวแทนของ limb	ยอมรับ	เพิ่ม IsAttached, controller, อีเวนต์ StateChanged และให้ RobotLimb เป็นที่อยู่ของกติกาของชิ้น (เลือดหมดแล้วหลุด ต่อกลับแล้วเติมเลือด) คำถาม "แขนซ้ายเป็นอย่างไร" ถามที่ robot.GetLimb(LeftArm) ที่เดียว ไม่เพิ่ม CanStrike / CanSupport ที่ RobotLimb เพราะเป็นค่าที่เปลี่ยนทุก tick ตามโหมดของตัวควบคุม และอยู่ใน FootContact กับ IStrikeSource แล้ว ในเกมนี้ "ใช้งานได้" เท่ากับ "ต่ออยู่" เพราะเลือดหมดแล้วหลุดทันที (RobotHealth.cs:192-195)
7	ความสัมพันธ์ระหว่าง RobotLimb กับ controller และวงจรชีวิตตอนหลุดไม่ชัด	ยอมรับ	RobotLimb ถือ controller : LimbController หนึ่งตัว และเพิ่มตารางวงจรชีวิตในรีวิวข้อ J: หลุดคือทำลาย joint อย่างเดียว (JointPullAndReconnect.cs:158) GameObject และ NetworkObject ยังอยู่ (แต่ละท่อนมี NetworkObject ของตัวเอง 15 ตัว ใน RobotContainer.prefab) ฟิสิกส์ยังรันบน server ขาที่หลุดยังรับ input และคลานตามจุดเล็ง (PlayerFootForRobot.cs:689-698, 822-835) แขนที่หลุดไม่ถูกขับและไม่รับ input (PlayerHandMovement.cs:281-284, 425) ลำตัวต้องเล็กน้อยเป็นเท้า ซึ่งโค้ดตอนนี้ยังน้อยอยู่ (หัวข้อ F1)
8	ArmController ทำทั้ง input, physics, grab และ network	ยอมรับบางส่วน	input ย้ายไป ArmInput อยู่แล้วในข้อเสนอเดิม รอบนั้นแยก ArmGrip เพิ่ม เพราะการจับมี state และโปรโตคอลขอ ยืนยัน และบังคับปล่อยของตัวเอง (PlayerHandMovement.cs:552-583, PlayerHandMovement.cs:607-761) และลำตัวต้องอ่านแค่ส่วนนี้ ไม่แยก "ชั้น network" ออกจากฟิสิกส์ เพราะใน NGO RPC ต้องเป็นเมธอดบน NetworkBehaviour ตัวรับเป็นบรรทัดเดียวที่ตรวจค่าแล้วเก็บเป้า (PlayerHandMovement.cs:585-605) และฟิสิกส์อ่านเป้าใน FixedUpdate ซึ่งคือ input → command → simulation อยู่แล้ว

#	ข้อวิจารณ์	ผล	คำตอบและสิ่งที่แก้
9	ชื่อ Controller หลอกลความหมาย ควรเป็น ArmMotor	เห็นต่าง	หลังย้าย input ออก คลาสนี้คือตัวควบคุมแบบป้อนกลับ (สปริง PD ที่ขับ rigidbody ไปหาเป้า) คำว่า Controller จึงตรงความหมาย และรื้อข้อ D นิยามคำศัพท์ไว้แล้ว (*Controller = ตัวขับฟิสิกส์ฝั่ง server, *Input = ตัวอ่าน input ของเจ้าของ) คำว่า Motor ขนกับ JointMotor ของ Unity ซึ่งเป็นอีกอย่างหนึ่ง (หัวข้อ E4)
10	LegController ทำทุกอย่างของการเคลื่อนที่	ยอมรับบางส่วน	ถูกว่าเราต้องมีโครง ตอนนี้อยู่ในโหมดของเท้าถูกตัดสินจากแฟล็ก bool 4 ตัว บวกสถานะลำตัวและท่าเตะ (PlayerFootForRobot.cs:422-491) ซึ่งเป็นที่มาของบั๊ก F1 แก่เป็น LegMode ค่าเดียวต่อ tick แต่ละโหมดมีเมธอดขับของตัวเอง และ FootContact คำนวณจากโหมด input ราว 300 บรรทัด (PlayerFootForRobot.cs:700-1002) ย้ายไป LegInput ไม่แยกเป็นคลาสเดิน กระโดด และยืน (หัวข้อ E7)
11	แขนกับขามีแพทเทิร์นซ้ำแต่ไม่มี abstraction กลาง	ยอมรับ	เพิ่ม LimbController (abstract) ที่มี rigidbody จุดหมุน สปริง การทำ joint chain ให้ไม่ซ้ำ และ ServerResetForRespawn() โค้ดที่ซ้ำจริงคือการเดินไข joint (PlayerHandMovement.cs:891-969 กับ PlayerFootForRobot.cs:289-307, 658-687) และการรีเซ็ตกับ RPC ล้างจุดเล็ง (PlayerHandMovement.cs:694-722 กับ PlayerFootForRobot.cs:1088-1127) และ Robot รีเซ็ตแขนขาทุกชิ้นได้ในรูปแบบเดียว
12	savedJoint ทำให้ LimbAttachment รุ้รายละเอียดมากเกินไป	เห็นต่าง	การต่อกลับต้องสร้าง ConfigurableJoint ใหม่ด้วยค่าเดิมทุกตัว ทั้ง anchor แกน drive และ limit (JointPullAndReconnect.cs:382-445) นี่คือหน้าที่หลักของคอมโพเนนต์นี้ และเป็นค่า private เห็นด้วยกับคำเตือนว่าอย่าให้มันโตเป็นระบบหลุดทั้งระบบ ซึ่งตอนนี้มันโตไปแล้ว (เขียนสถานะของตัวควบคุมและสิ่งลำตัวล้ม JointPullAndReconnect.cs:173-184) ข้อเสนออย่าสอยอย่างนี้ไปที่ RobotLimb และ Robot
13	ไม่ชัดว่า HP = 0 แล้วเกิดอะไร	ยอมรับ	แผนภาพเดิมขาดจริง กติกาจากโค้ดคือ HP = 0 → เพดานเลือดลด 125 (ต่ำสุด 50) → ขึ้นหลุด (RobotHealth.cs:218-228) ขึ้นที่หลุดไม่รับดาเมจ (RobotHealth.cs:177) และต่อกลับแล้วเลือดเต็มตามเพดานใหม่ (RobotHealth.cs:72-81) ตอนนี้นำมาไว้ในแผนภาพ 3 และ 11 โดย RobotLimb เป็นผู้ implement IDamageable แทน LimbHealth เพื่อให้ขึ้นที่หลุดปิดดาเมจทั้งได้เหมือนเดิม
14	LimbStrike ผูกกับ Collision และ IStrikeSource ลอยอยู่	ยอมรับบางส่วน	OnCollisionEnter ต้องอยู่บนอ็อบเจกต์ที่มี collider LimbStrike คือตัวแปลงการชนของ Unity เป็น DamageInfo การฟัง Collision จึงอยู่ถูกที่แล้ว ผู้ implement IStrikeSource อยู่ในแผนภาพ 5 (PunchAbility, KickAbility) และแผนภาพ 8 (SwordHeldItem) และสายเดิมอยู่ในแผนภาพ 6 และ 9 แผนภาพ 3 ใหม่ไม่ว่า LimbStrike ซ้ำ จะได้ไม่มีอินเทอร์เฟซที่ไม่เห็นผู้ implement ในภาพ
15	ไม่ชัดว่าตีลำตัวหรือข้อต่อแล้วดาเมจไปไหน	ชี้แจงและเพิ่มในแผนภาพ	ลำตัวไม่มีเลือดโดยตั้งใจ เพราะไม่มี joint ให้หลุด ดาเมจที่ลำตัวลงชั้นที่ใกล้จุดปะทะที่สุดที่ยังต่ออยู่ เพื่อไม่ให้ทั้ง 4 ชั้นเลือดหมดพร้อมกัน (PvpRobotTeam.cs:161-196) ในข้อเสนอคือ RobotBodyDamageRelay ซึ่งตอนนี้อยู่ในแผนภาพ 3 ด้วย ตีข้อต่อก็คือโดน collider ของชั้นนั้น
16	LimbSlot เป็น enum อาจไม่พอเมื่อระบบโต	เห็นต่าง	topology 4 ชั้นคือแก่นของเกม (หัวข้อ E5) และโค้ดนิยามช่องแขนขาซ้ำกันอยู่แล้วหลายที่ (LimbStatusUI.LimbSlot กับ PvpLimb แบบ 4 ช่อง และ TutorialManager.LimbRole แบบแขนหรือขา) ซึ่งรื้อรวมเป็น enum ตัวเดียว ข้อวิจารณ์เองก็บอกว่า enum พอสำหรับหุ่นแบบตายตัว
17	RobotLimb[4] เป็น topology แบบ hard-code	เห็นต่าง	array ที่ index ด้วย LimbSlot คือชนิดที่ตรงที่สุดสำหรับ topology ตายตัว ส่วน List<RobotLimb> ทำให้ "หุ่นไม่มีขาซ้าย" เขียนได้ในชนิดข้อมูล ทั้งที่ชั้นที่หลุดยังเป็นของหุ่น การหลุดเป็นสถานะ ไม่ใช่การถอดชิ้นออกจากหุ่น

#	ข้อวิจารณ์	ผล	คำตอบและสิ่งที่แก้
18	owner : Robot ทำให้อ้างอิงสองทาง	เห็นต่าง	ทั้งสองทางถูกตั้งใน prefab และไม่เปลี่ยนตอนรัน ขึ้นต่อกลับได้เฉพาะ socket เดิมของหุ่นตัวเอง (JointPullAndReconnect.cs:123-139) หุ่นกับแขนขาเกิดและหายพร้อมกันเพราะเป็น prefab เดียว จึงไม่มีการผูกตอนรันที่ต้องห่วงเรื่องลำดับ วงจรชีวิตเดียวที่มีคือสถานะต่ออยู่ ซึ่ง LimbAttachment เป็นเจ้าของ
19	ยังไม่ได้ model ว่าใครเป็น authority	ยอมรับบางส่วน	มีในแผนภาพ 5 และรีวิซ G อยู่แล้ว แต่แผนภาพ 3 ไม่บอก และไม่มีการตั้งตั้งแต่ input ถึงจว ข้อเท็จจริงคือฟิสิกส์รันบน server เท่านั้น (FixedUpdate ของลำตัวและแขนขาขึ้นต้นด้วย if (!IsServer) return) NetworkTransform ใน prefab หุ่นเป็น Server ทั้งหมด และไม่มีการทำนายผลฝั่ง client เพิ่มตาราง authority ในรีวิซ G และแผนภาพ 12 และเจอช่องโหว่จริงหนึ่งจุด: RPC ของแขนขาใช้สิทธิ์ค่าเริ่มต้นที่ใครก็เรียกได้ (หัวข้อ F4)
20	RPC เยอะจนขึ้น network ฝั่งในทุก component ควรเป็น Input → Command → Simulation → State	เห็นต่าง	RPC ของแขนขาคือขึ้น command อยู่แล้ว เป็นคำสั่งที่มีชื่อ server ตรวจค่า (ValidateAndSetFootTarget) แล้วเก็บไว้ ฟิสิกส์อ่านใน FixedUpdate และผลกลับไปทุกเครื่องผ่าน NetworkTransform กับ NetworkVariable ส่วนสิ่งที่ยอมรับคือตัวรับ RPC ต้องบางและเป็น owner-only (หัวข้อ E3)
21	เมธอด Server... เติมไปหมดทุก object กลายเป็น network endpoint	เห็นต่าง	เมธอด Server* ไม่ใช่ network endpoint แต่เป็นเมธอด C# ธรรมดาที่ตกลงกันว่าเรียกบน server เท่านั้น (รีวิซ D) endpoint จริงมีแค่เมธอด *Rpc เรื่องการเทส ลอจิกที่ควรเทสย้ายไปคลาสธรรมดาแล้ว (FallRules และกติกาใน DamageRouter) เทสใน EditMode ได้แบบเดียวกับ Assets/Editor/ItemAuthorityTests.cs
22	data กับ logic ยังปนกัน	ชี้แจงและปรับแผนภาพ	แยกตามชนิดไว้แล้ว StrikeTuning เป็น ScriptableObject (ค่าตั้ง) DamageInfo เป็น struct (ค่าต่อการโดนหนึ่งครั้ง) ArmMotionOverride เป็น struct พารามิเตอร์ต่อ tick และ SavedJointSettings เป็น struct ภายในตอนรัน ภาพรวมอยู่ในแผนภาพ 14 แผนภาพใหม่ใส่ stereotype ให้ครบ และแยก FallLimits (ค่าตั้ง) ออกจากตัวจับเวลาของ FallRules
23	TorsoState หยิบไป ควรมี Unstable, Staggering, Recovering, Disabled	เห็นต่าง (ในทิศตรงข้าม)	โค้ดมีพฤติกรรมแค่สองแบบคือยืนกับล้ม Falling อยู่ไม่เกิน 1 FixedUpdate (TorsoMovement.cs:174-176) และทั้ง 8 จดนอก TorsoMovement ที่อ่านค่านี้ถือว่าเท่ากับ Ragdoll เช่น PlayerHandMovement.cs:555, PlayerFootForRobot.cs:425, StatusPromptUI.cs:87-88 จึงลดเหลือ {Standing, Ragdoll} บวก FallReason (หัวข้อ E2)
24	stress ไม่ใช่ NetworkVariable อาจ desync	ชี้แจงและปรับแผนภาพ	stress ถูกเขียนและอ่านใน FixedUpdate ฝั่ง server เท่านั้น (TorsoMovement.cs:127, 157, 479 และ PlayerHandMovement.cs:803, 859 ซึ่งรันได้ IsServer) client ไม่ได้ตัดสินการล้ม แค่อ่าน state จึง desync ไม่ได้ แผนภาพใส่ {server only} แล้ว
25	OnLimbAttachmentChanged (bool) ไม่บอกว่าขึ้นไหน	ยอมรับ	อีเอนต์เดิมเป็น Action<bool> (JointPullAndReconnect.cs:41) ซึ่งพอสำหรับภายในชั้น แต่ Robot ฟัง 4 ชั้น เปลี่ยนเป็น RobotLimb.StateChanged(RobotLimb) และ Robot.OnLimbStateChanged(limb : RobotLimb)
26	naming ปนกันหลายแบบ	ยอมรับบางส่วน	คำนำหน้าเป็นแบบแผนที่ความหมายต่างกัน (Server* = เรียกบน server, *Rpc = NGO บังคับ, On* = ตัวรับอีเอนต์, Request* = client ขอ) กริยาต่างกันเพราะการกระทำต่างกัน ที่แก้คือจุดที่ไม่สม่ำเสมอจริง: ลบ ReportArmPull, เปลี่ยน ServerRegisterFootJump เป็น ServerApplyJump และ UpdateHandTargetRpc / UpdateFootTargetRpc เป็น SetHandTargetRpc / SetFootTargetRpc
27	ReportArmPullIntensity เฉพาะแขนเกินไป	ยอมรับ	และเจอบั๊ก: armPullIntensity เป็นฟิลด์ public ที่มีทุกข้างเขียนทุก tick (PlayerHandMovement.cs:805, 809) มือที่ไม่ได้จับเขียน 0 หับมือ

#	ข้อวิจารณ์	ผล	คำตอบและสิ่งที่แก้
			ที่ปีนอยู่ (หัวข้อ F2) แก้: ไม่มี API ให้แขนส่งค่าเข้ามา ลำตัวอ่าน ClimbPull ของแขนที่ต่ออยู่แล้วใช้ค่ามากที่สุด (ต่างจากตอนนี้นี้เล็กน้อย ต้องลองเปรียบเทียบ) ไม่ใช่ชื่อ ReportSupportForce เพราะแรงป็นไม่ใช้การพุง แต่เป็นการลดแรงดึงเข้ากลาง (TorsoMovement.cs:308)
28	RegisterFoot ทำให้ลำตัวเป็น registry และไม่มี Unregister	อ่านแผนภาพผิด แต่ประเด็นจริงในโค้ด	เมธอดในแผนภาพคือ ServerRegisterFootJump แปลว่าเท้าเพ็งกระโดด (หน้าต่างกระโดดสองคน TorsoMovement.cs:395-418) ไม่ใช่การลงทะเบียน แต่ชื่อนี้ชวนอ่านผิดจริง จึงเปลี่ยนเป็น ServerApplyJump(LimbSlot) ส่วนในโค้ดมี registry จริง (TorsoMovement.cs:105-108) ที่ลงทะเบียนตอน spawn และถอนตอน despawn เท่านั้น (PlayerFootForRobot.cs:219, 340) ไม่ถอนตอนหลุด ข้อเสนอเอา registry ออกทั้งหมด
29	หลุดแล้วการลงทะเบียนเป็นอย่างไร	ยอมรับ	ไม่มีการลงทะเบียนให้แก้แล้ว ลำตัวสร้าง BalanceInput จากแขนขาทั้ง 4 ขึ้นทุก tick ส่วนจะนับขาที่หลุดหรือไม่เป็นค่าใน FallLimits ค่าเริ่มต้นนับแบบเดิม เพื่อไม่ให้ความรู้สึกตอนเหลือขาเดียวเปลี่ยนโดยไม่มีใครตัดสิน (หัวข้อ F1) ลำดับเต็มอยู่ในแผนภาพ 11
30	respawn กระจายไปทุก component ไม่มี RespawnSystem	ยอมรับบางส่วน	ถูกว่าต้องเห็นลำดับ เพิ่มแผนภาพ 13 และเจอปัญหาจริง: RespawnManager รู้จักทุกคลาสของหุ่นและแคช joint ไว้ตอน Awake (RespawnManager.cs:109-125) ขณะที่การต่อกลับสร้าง joint ใหม่ (JointPullAndReconnect.cs:258) (หัวข้อ F3) ข้อเสนอ: RespawnManager ตัดสินว่าเมื่อไหร่และที่ไหน ส่วน Robot. ServerResetForRespawn ตัดสินว่าอย่างไร ไม่เห็นด้วยที่ว่าชิ้นส่วนไม่ควรมีเมธอดรีเซ็ตของตัวเอง เพราะแต่ละชิ้นมี state ส่วนตัว (จุดปักเท้า joint ที่จับ ตัวจับเวลา) ถ้าให้คนอื่นล้างแทนจะทำลาย encapsulation การต่อแขนขาคืนและเติมเลือดตอน respawn ไม่ได้ทำในโค้ดตอนนี้ เป็นเรื่องที่ต้องเลือก (หัวข้อ H)
31	FromCollider มีปัญหาเมื่อ hierarchy ซับซ้อน	ยอมรับบางส่วน	ถูกสำหรับวิธีที่ใช้ตอนนี้ RobotBodyCheck ใช้ transform.root (NetworkCheck.cs:41) ซึ่งฟังถ้าหุ่นถูกวางใต้อ็อบเจกต์อื่น กำหนดให้ FromCollider คือ hit.attachedRigidbody. GetComponentInParent<Robot>() โดย Robot อยู่บน root ของ prefab หุ่น (แขนขาเป็นลูกพี่น้องกันได้ root NetworkCheck.cs:29-30) ชิ้นที่หลุดยังอยู่ใต้ root เดิม จึงยังหาเจ้าของเจอ ซึ่งถูกสำหรับการเช็คทิม ส่วนดาเมจ LimbHealth ปฏิเสธชิ้นที่หลุดเอง
32	composition กับ aggregation ไม่ชัดเรื่องอายุของอ็อบเจกต์	ชี้แจงและแก้การวาด	legend ในรีวิวนิยามไว้แล้ว Robot กับ RobotLimb อยู่ใน prefab เดียวกัน เกิดและหายพร้อมกัน และหลุดไม่เท่ากับทำลาย ส่วนเส้น RobotLimb → ตัวควบคุมที่เคยวาดเป็น aggregation (ข้าวหลามตัดโปร่ง) ผิดจริง เพราะอายุเท่ากัน แก้เป็น composition แล้ว
33	LimbAttachment กับ RobotLimb ใครเป็นเจ้าของความจริง	ยอมรับ	เจ้าของคือ LimbAttachment.isAttached (NetworkVariable) ตัวเดียว RobotLimb.isAttached แค่อ่านต่อ และไม่เขียนสถานะขึ้นนอกจากส่ง ServerDetach() ตามกติกา โค้ดตอนนี้มีสองแหล่งจริง (HandState/FootState กับ netIsConnected) ซึ่งรีวิวนเสนอให้ลบ HandState/FootState (รีวิข้อ B แถว 6)
34	ความสัมพันธ์ระหว่าง LimbHealth กับ LimbAttachment ไม่แสดง	ยอมรับ	เหมือนข้อ 13 ตอนนี้สองตัวไม่รู้จักกัน RobotLimb เป็นคนเชื่อม (แผนภาพ 3 และ 11)
35	damage pipeline ไม่ครบ	ชี้แจง	อยู่ในแผนภาพ 6 และ 9 DamageRouter เป็นตัวตัดสิน (เพลสของแมตซ์และกติกาทีม) DamageInfo เป็นข้อมูล และผู้รับ 5 ชนิด implement IDamageable ไม่เพิ่ม Processor หรือ Resolver อีก เพราะตัวตัดสินตัวเดียวพอสำหรับผู้รับ 5 ชนิด

#	ข้อวิจารณ์	ผล	คำตอบและสิ่งที่แก้
36	Team ไม่มีใครใช้ในแผนภาพ	ชี้แจง	ใช้ใน DamageRouter ผ่าน IDamageable.Team() กับ DamageInfo.attackerTeam (แผนภาพ 6) และใน PvpModeRules (แผนภาพ 7) ทีมอยู่ที่ Robot เพราะใน PvP ทีมเป็นของหุ่นทั้งตัว
37	ไม่แยก physical limb กับ logical limb	เห็นต่าง	เกมนี้ฟิสิกส์คือเกมเพลย์ การแยกที่มีประโยชน์มีแล้ว คือ RobotLimb (ตัวตนและกติกา) LimbAttachment (joint) และ LimbController (ขับ rigidbody) (หัวข้อ E4)
38	controller ผูกกับ RigidBody ตรงเกินไป	เห็นต่าง	การขับ RigidBody คือหน้าที่ของตัวควบคุม ดูข้อ 9 และ 37
39	ไม่มี LimbRole หรือ capability	เห็นต่าง	ข้อวิจารณ์เองก็บอกว่าไม่จำเป็นสำหรับหุ่นแบบตายตัว ความสามารถของชิ้นบอกด้วยคอมโพเนนต์ที่ติดอยู่ (ArmGrip, PunchAbility, KickAbility) และ LimbSlot.IsArm() / IsLeg() สำหรับ UI
40	แผนภาพขาด flow	ยอมรับบางส่วน	ชุดแผนภาพมี sequence และ data flow อยู่แล้ว (แผนภาพ 9, 10, 14) แต่ขาดสามเรื่องที่ข้อวิจารณ์ยกมาจริง ตอนนี้เพิ่มแล้ว: เลือดหมด → หลุด → สัม → ต่อกลับ (แผนภาพ 11) ก้าวเท้าจาก input ถึงจอกทุกคน (แผนภาพ 12) และ respawn (แผนภาพ 13)

E. ข้อที่เห็นต่าง อธิบายเต็ม

E1. อินเทอร์เฟซ support แบบรวม (ข้อ 2)

ข้อวิจารณ์เสนอ ISupportProvider ให้เขาและแขน implement เพื่อให้ลำตัวไม่ต้องรู้ว่าข้างหลังเป็นอะไร และรองรับหาง ล้อ หรือส่วนพยางอื่นในอนาคต

กฎในโค้ดแยกการพยางสองแบบออกจากกันโดยตั้งใจ:

- มือที่จับของนิ่ง (kinematic) กันการล้มได้ทุกกฎ รวมกฎ stress (TorsoMovement.cs:157 เช็ค !supportedByHand)
- ขาที่ยืนพื้นกันได้เฉพาะกฎการทรงตัว (เอียง ลอย เสียสมดุล) ผ่าน balanceProtected (TorsoMovement.cs:224) แต่ไม่กันกฎ stress
- กฎหลักของเกมคือ "สองขาก้าวพร้อมกัน" (TorsoMovement.cs:249) ซึ่งเป็นเรื่องของขาโดยเฉพาะ และคอมเมนต์ในโค้ดบอกว่านี่คือความยากที่ตั้งใจ (TorsoMovement.cs:245-248)

ถ้ารวมเป็น ISupportProvider ตัวเดียว ลำตัวจะต้องถามกลับว่า support ขึ้นนี้เป็นแบบไหน ซึ่งก็คือการรู้จักชนิดเดิมผ่านอีกชั้นหนึ่ง ถ้าแยกเป็นสองอินเทอร์เฟซ แต่ละตัวจะมีผู้ implement ตัวเดียว ซึ่งขัดกฎของรีวิว ส่วนหางหรือล้อไม่อยู่ในการออกแบบของเกมที่ผู้เล่น 4 คนคุม 4 ชั้น

สิ่งที่รับจากข้อนี้คือปัญหาที่อยู่ข้างใต้ คือรายชื่อกำลังเหวี่ยงไวและไม่ตามสถานะการหลุด ซึ่งแก้ด้วยการอ่านจาก Robot ทุก tick และส่งสถานะเป็นค่า (FootContact) ให้ FallRules เผลได้โดยไม่ต้องมี MonoBehaviour

E2. state ของลำตัวควรมีมากขึ้น (ข้อ 23)

state machine ควรมี state เท่ากับจำนวนพฤติกรรมที่ต่างกันจริง โค้ดตอนนี้มีพฤติกรรมสองแบบ คือยืน (มีแรงพยาง ขาปักพื้น มือดึงลำตัวได้) กับล้ม (แรงพยางหาย มือห้อย ขาอ่อน กด Q เพื่อลุก) Falling ไม่มีพฤติกรรมของตัวเอง มันอยู่ไม่

เกินหนึ่ง FixedUpdate แล้วกลายเป็น Ragdoll (TorsoMovement.cs:174-176) และทุกจุดที่อ่านค่าถือว่ามันเท่ากับ Ragdoll

การเพิ่ม Unstable, Staggering หรือ Recovering ตอนนี้จะสร้าง state ที่ไม่มีโค้ดไหนทำอะไรต่างออกไป และเพิ่มจุดที่ต้องเช็คในแปดที่ที่อ่านค่านี้ ข้อเสนอจึงไปทางตรงข้าม คือเหลือสองค่า แล้วเก็บ "ล้มเพราะอะไร" ไว้ใน FallReason ถ้าวันหนึ่งนักออกแบบเพิ่มท่าเซที่มีแอนิเมชันหรือกติกาของตัวเอง ค่อยเพิ่ม state ตอนนั้น

E3. ชั้น command แยก และเมธอด Server* (ข้อ 20, 21)

ลำดับที่ข้อวิจารณ์เสนอ (Input → Command → Server Simulation → Network State) คือสิ่งที่โค้ดทำอยู่แล้ว เจ้าของอ่าน input ใน Update() ส่งคำสั่งที่มีชื่อเป็น RPC server ตรวจสอบแล้วเก็บ ฟิสิกส์อ่านคำสั่งล่าสุดใน FixedUpdate และผลกลับไปทุกเครื่องผ่าน NetworkTransform กับ NetworkVariable (แผนภาพ 12)

ชั้น command ที่แยกเป็นระบบ (บัฟเฟอร์คำสั่ง หมายเลข tick การยืนยัน) คุ่มเมื่อต้องทำนายผลฝั่ง client แล้ว rollback ซึ่งกับหุ่นที่ต่อด้วย joint และฟิสิกส์ต่อเนื่องทำได้ยากมาก และเกม co-op ที่ทุกคนเล่นหุ่นตัวเดียวกันก็ทนได้เลยได้ จึงไม่ใช่ปัญหาที่เกมนี้มี

ส่วนเมธอด Server* ไม่ใช่ network endpoint เป็นแค่แบบแผนการตั้งชื่อที่เรียกบน server เท่านั้น endpoint จริงคือ เมธอด *Rpc ซึ่งจะเป็น owner-only ทุกตัว

E4. ชั้นฟิสิกส์แยก และชื่อ Controller (ข้อ 9, 37, 38)

ILimbPhysics จะมีผู้ implement ตัวเดียวคือ PhysX ของ Unity ไม่มีระบบฟิสิกส์อื่นให้สลับ และพฤติกรรมของแขนขา ก็ทดสอบโดยใช้ฟิสิกส์ไม่ได้อยู่ดี โปรเจกต์ใช้ harness _ArmFeelTest สำหรับวัดฟิสิกส์ของแขนในฉากจริง ซึ่งเป็นวิธีที่ถูกสำหรับเรื่องนี้

การแยกที่มีประโยชน์ในเกมนี้มีแล้ว คือตัวตนและกติกาของชิ้น (RobotLimb) joint (LimbAttachment) และการขับ rigidbody (LimbController) ส่วนลอจิกที่ไม่ใช่ฟิสิกส์และควอเตอร์ (กฎการล้ม กติกาตาเมจ) แยกเป็นคลาสธรรมดาแล้ว ชื่อ Controller ถูกความหมายหลังย้าย input ออก เพราะคลาสนี้คือตัวควบคุมแบบป้อนกลับที่คำนวณแรงเพื่อให้ rigidbody ไปถึงเป้า ถ้าเปลี่ยนเป็น Motor จะชนกับ JointMotor ของ Unity ซึ่งเป็นอีกกลไกหนึ่ง

E5. topology แบบยืดหยุ่น และ capability (ข้อ 16, 17, 39)

เกมนี้ออกแบบให้ผู้เล่น 4 คนคุมแขนขาคนละชิ้น กฎการล้มนับสองขา การกระโดดสองคนต้องการสองขาจากคนละคน (TorsoMovement.cs:403-404) ลอบบี้ HUD และ PvP ใช้ 4 ช่อง และกติกา PvP แพ้เมื่อหลุดครบ 4 ชิ้น หุ่นที่มีหางหรือแขนที่สามไม่ใช่การต่อยอดระบบนี้ แต่เป็นเกมอีกแบบที่ต้องออกแบบกติกาใหม่หมด การทำให้โค้ดรองรับไว้ก่อนจะทำให้ทุกที่ต้องรับมือกับกรณีที่ไม่มีความเกิด เช่น "หุ่นมีขาเดียวตั้งแต่เกิด"

E6. Robot รับงานเยอะ และ FromCollider (ข้อ 4, 5)

ทุกเมธอดของ Robot เป็นงานของหุ่นทั้งตัว: หาชิ้นส่วน เปิดปิดการจำลอง รีเซตตอน respawn และตอบสนองเมื่อชิ้นหลุด ถ้าย้ายออกไปคลาสอื่น คลาสนั้นก็ต้องรู้จักชิ้นส่วนทั้งหมดของหุ่นอยู่ดี ซึ่งคือสิ่งที่ RespawnManager ทำอยู่ตอนนี้และเป็นที่มาของปัญหา F3

FromCollider เป็นเมธอด static ที่ตอบคำถาม "collider นี้เป็นของหุ่นตัวไหน" ซึ่งระบบทีม ไอเทม บอส และหลุมดำต้องถาม และตอนนี้แต่ละที่ถามด้วยวิธีของตัวเอง การที่มันอ้าง Collider ไม่ใช่การรู้ เพราะหุ่นเป็นคอมโพเนนต์ของ Unity อยู่แล้ว ส่วนเส้นทางดาเมจไม่ผ่าน Robot เลย ตรงกับที่ข้อวิจารณ์ต้องการ

E7. แยก LegController เป็นหลายคลาส (ข้อ 10)

การยืน การก้าว การกระโดด และการย่นตัวลุก ใช้ rigidbody เป้า และจุดปักเดียวกันทุก tick และสลับกันตามสถานะของลำตัวกับท่าเตะ ถ้าแยกเป็นคลาสต่อพฤติกรรม คลาสเหล่านั้นต้องแย่งกันคุม rigidbody ตัวเดียว ซึ่งโค้ดเจอปัญหานี้อยู่แล้วระหว่างทำท่าเตะ ต้องมี IsKickControllingFoot ไว้บอกว่าใครคุม (PlayerFootForRobot.cs:207-213) ทางที่ดีกว่าคือทำให้โหมดชัดเจนเป็นค่าเดียวต่อ tick (LegMode) แล้วให้แต่ละโหมดมีเมธอดขับของตัวเอง

F. ปัญหาที่เจอระหว่างตรวจ

ทั้งสี่ข้อมาจากการอ่านโค้ด ยังไม่ได้ลองใน Play Mode และถูกเพิ่มในรีวิวข้อ B แถว 25-28 แล้ว

F1. ลำตัวยังนับขาที่หลุดเป็นเท้า (High)

- เท้าลงทะเบียนกับลำตัวตอน spawn (PlayerFootForRobot.cs:219) และถอนตอน despawn เท่านั้น (PlayerFootForRobot.cs:340) ไม่ถอนตอนหลุด
- ลูปของลำตัวนับทุกเท้าที่ลงทะเบียน (TorsoMovement.cs:196-213) และ IsGrounded() เช็คแค่ capsule จากสะโพกถึงเท้า ไม่เช็คควาต่ออยู่ (PlayerFootForRobot.cs:1074-1080)
- ผลที่หนึ่ง: ขาที่หลุดนอนบนพื้นถูกนับว่าแตะพื้นและสมดุล แรงดึงลำตัวเข้ากลาง (TorsoMovement.cs:304-309) จึงดึงไปทางขาที่หลุด และกฎ "ลอยนานเกิน" ไม่ทำงาน
- ผลที่สอง: ถ้าขาหลุดตอนผู้เล่นกดคลิกซ้ายค้าง isStepping ของขานั้นค้างเป็น true บน server เพราะถูกล้างเฉพาะในสาขาที่ต่ออยู่ของ PlayerFootForRobot.HandleInput (PlayerFootForRobot.cs:710-715, PlayerFootForRobot.cs:800-805) พอลุกขึ้นแล้วขาที่เหลือก้าวเมื่อไหร่ กฎสองขาก้าวพร้อมกันจะนับครบสองขาแล้วสั่งล้มใน 0.15 วินาที
- วิธีลอง: ใช้ปุ่มดีบักทำให้ขาหลุดขณะกดคลิกซ้ายค้าง กด Q ลุกด้วยขาที่เหลือ แล้วก้าว
- ในข้อเสนอ: ไม่มีรายชื่อ ลำตัวอ่านแขนขาจาก Robot แต่ผลทั้งสองข้อข้างบนเปลี่ยนความรู้สึกของเกม ข้อเสนอจึงเก็บแบบเดิมไว้เป็นค่าเริ่มต้น (FallLimits.countDetachedLegs = true และแฟล็ก stepping ไม่ถูกล้างตอนหลุด) ขาที่หลุดยืนพื้นแทนขาไม่ได้เหมือนตอนนั้น ถ้าทีมลองเล่นแล้วตัดสินใจเป็นบั๊ก ปิดได้ที่ FallLimits กับ LegController ที่เดียว (โน้ตสีส้มในแผนภาพ 4 และ 5)

F2. ค่าแรงป็นถูกเขียนทับ (Medium)

- armPullIntensity เป็นฟิลด์ public ของลำตัว มือที่ป็นเขียนค่าจริง (PlayerHandMovement.cs:805) มือที่ไม่ได้จับเขียน 0 (PlayerHandMovement.cs:809) และทั้งสองมือรันทุก FixedUpdate
- ค่าที่ลำตัวใช้ลดแรงดึงเข้ากลางตอนปีน (TorsoMovement.cs:308) จึงเป็นของแขนที่รันทีหลัง ไม่ใช่ของแขนที่ปีน
- วิธีลอง: จับขอบนั่งด้วยแขนข้างเดียวขณะอีกข้างว่าง แล้ว log ค่านี้ใน TorsoMovement.FixedUpdate
- ในข้อเสนอ: ลำตัวอ่าน ClimbPull ของแขนที่ต่ออยู่ทุกข้างแล้วใช้ค่ามากที่สุด แบบเดิมเก็บไว้ไม่ได้เพราะผลขึ้นกับลำดับสคริปต์ จึงต้องลองปีนเทียบก่อนใช้

F3. respawn ฟังก์ชัน joint ที่แคชไว้ตอนเริ่มฉาก (Medium)

- RespawnManager เก็บ joint ทั้งหมดของหุ่นครั้งเดียวตอน Awake (RespawnManager.cs:109-125) เพื่อปลดข้อครหาเรื่อง teleport

- การต่อแขนขากลับสร้าง `ConfigurableJoint` ตัวใหม่ (`JointPullAndReconnect.cs:258`) ที่ไม่อยู่ในแคช แขนขาที่เคยหลุดแล้วต่อกลับจึงไม่ถูกปลดระหว่าง teleport
- วิธีลอง: ทำให้แขนหลุด ดึงกลับมาต่อ แล้วเดินตกแมพ ดูว่าแขนข้างนั้นกระตุกหรือยืดตอนเกิดใหม่ต่างจากข้างที่ไม่เคยหลุดไหม
- ในข้อเสนอ: `Robot.ServerResetForRespawn` อ่าน joint ที่ต่ออยู่ตอนนั้นจากแขนขา (แผนภาพ 13)

F4. RPC ของแขนขาไม่ได้จำกัดเจ้าของ (Medium)

- RPC ของมือ เท้า และลำตัวประกาศเป็น `[Rpc(SendTo.Server)]` โดยไม่ระบุสิทธิ์ (`PlayerHandMovement.cs:585-586, 607, 663, PlayerFootForRobot.cs:1008-1015, TorsoMovement.cs:465`)
- ใน NGO 2.12 ค่าเริ่มต้นของ `InvokePermission` คือ `Everyone` (`RpcAttributes.cs:91-93` ในแพ็คเกจ) client ไหนก็ส่งเป้าหมายให้แขนขาของคนอื่นได้ ใน PvP คือขับแขนขาของฝ่ายตรงข้ามได้ถ้า client ถูกแก้
- `PlayerInventory` ทำถูกแล้วด้วย `RpcInvokePermission.Owner` แก่แบบเดียวกัน

G. ลำดับความสำคัญหลังตรวจโค้ด

ลำดับ	เรื่อง	ข้อ	เหตุผล
1	วงจรชีวิตของขาที่หลุด และการเลิกใช้ registry	7, 28, 29	มีบั๊กที่น่าจะเกิดจริงกับกติกาหลักของเกม (F1)
2	สิทธิ์ของ RPC	19	ช่องโหว่ใน PvP แก้ง่าย (F4)
3	<code>FallRules</code> + <code>FallReason</code> และผู้สั่งล้มคนเดียว	1, 3, 23	กติกาหลักต้องทดสอบได้ และต้องรู้ว่าล้มเพราะอะไร
4	ค่าที่หลายคนเขียนทับ	27	บั๊ก F2
5	respawn ที่ฝั่งแคช	30	บั๊ก F3 และ <code>RespawnManager</code> รู้จักทุกคลาสของหุ่น
6	กติกาเลือดกับการหลุดผ่าน <code>RobotLimb</code>	13, 34	ตัดการที่ health กับ attachment อ้างกันเอง
7	โครงของตัวควบคุม: <code>LimbController</code> , <code>ArmGrip</code> , <code>LegMode</code>	8, 10, 11	ลดโค้ดซ้ำ และทำให้โหมดของขาชัด
8	ความชัดของแผนภาพ	14, 22, 24, 25, 32	signature, stereotype, server only และชนิดของเส้น

ที่ไม่ทำ: อินเทอร์เฟซ support แบบรวม, state ของลำตัวที่มากขึ้น, ชั้น command แยก, ชั้นฟิสิกส์แยก, topology แบบยืดหยุ่น, ระบบ capability, การเปลี่ยนชื่อ Controller และการย้าย `FromCollider` ออกจาก `Robot`

เทียบกับลำดับในข้อวิจารณ์ ข้อที่ตรงกันคือเรื่องลำตัว วงจรชีวิตตอนหลุด ความสัมพันธ์ของเลือดกับการหลุด และ respawn ข้อที่ต่างคือ "Collision รัวเข้า Robot" กับ "damage pipeline ไม่ครบ" ซึ่งไม่ใช่ปัญหาเมื่อดูทั้งชุดแผนภาพ และมีสองเรื่องข้อวิจารณ์ไม่ได้พูดถึงแต่ขึ้นมาย่ออันดับต้น คือสิทธิ์ของ RPC และค่าที่หลายคนเขียนทับ

H. เรื่องที่ทีมต้องตัดสินใจ

- หุ่นขาเดียวควรยืนและเดินได้แค่ไหน พฤติกรรมตอนนี้ขึ้นกับการนับขาที่หลุด (F1) พอแก้แล้วหุ่นขาเดียวที่ยกขาจะไม่ล้มเฝ้าแต่พื้น และล้มตามกฎ "ลอยนานเกิน" ใน 0.5 วินาที (`fallGracePeriod` ใน `RobotContainer.prefab`) ถ้าต้องการให้เดินขาเดียวได้ ต้องเพิ่มกติกานี้ใน `FallRules` อย่างตั้งใจ

- **respawn** การต่อแขนขาที่หลุดคืนและเติมเลือดใหม่ ตอนนี้ไม่ทำ ถ้าจะทำก็เพิ่มใน `Robot`.
`ServerResetForRespawn` ที่เดียว
- **RPC** ที่ต้องให้คนที่ไม่ใช่เจ้าของเรียก จากที่ตรวจมีแค่ปุ่มดับก ซึ่งควรคอมไพล์เฉพาะใน Editor และ development build อยู่แล้ว (รีวิวข้อ B แถว 10)